Universidad Corporativa Coppel | Programas Académicos

Ingeniería en Desarrollo de Software

Actividad | #3 | Backend para el Sistema Web.

Desarrollo de Sistemas Web II

Ingeniería en Desarrollo de Software

academia global

TUTOR: Dr. Rafael Ceballos.

ALUMNO: Manuel Marin Sanchez

FECHA: 13/10/2026.

Índice.

Portada 1
Índice 2
Introducción 3
Investigación 4
Desarrollo 5,6,7
Conclusiones 8
Referencias bibliográficas 9

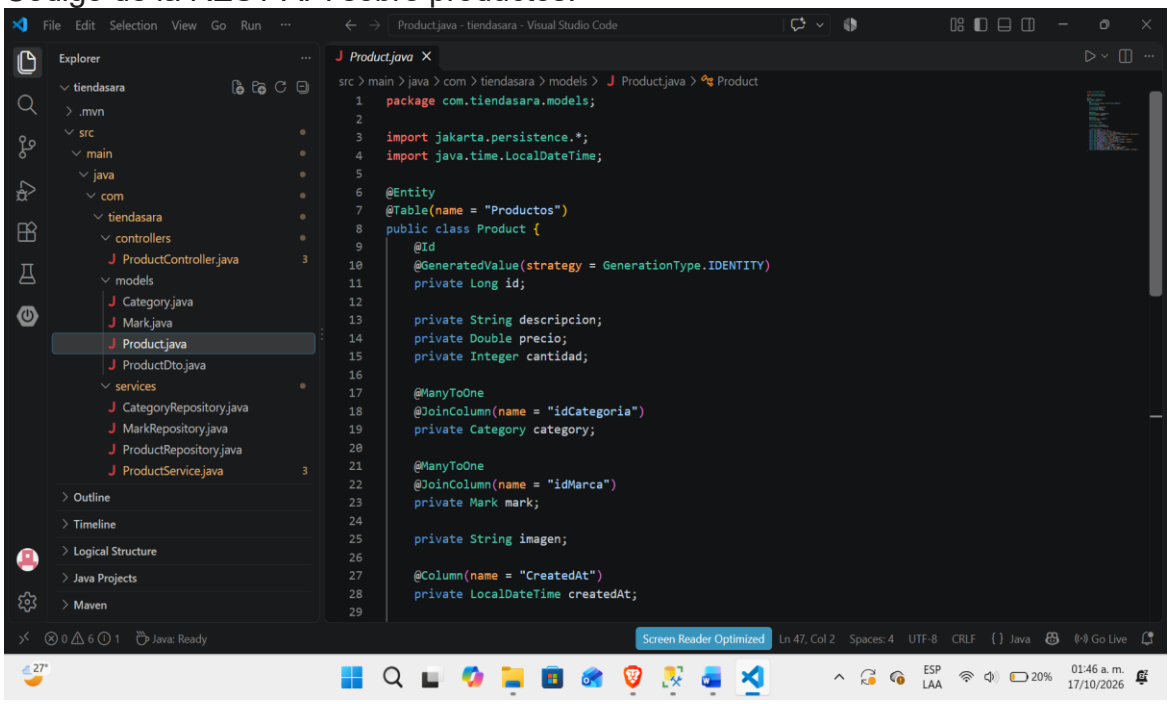
Introducción.

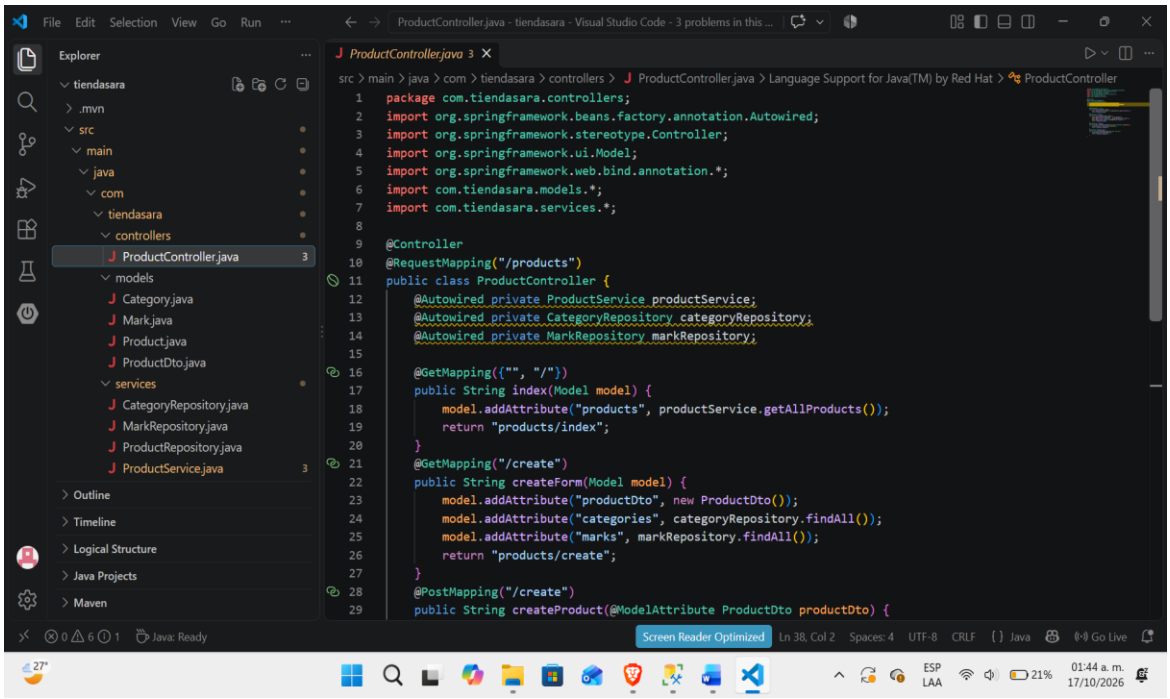
En esta tercera etapa del proyecto, el enfoque se traslada desde la interfaz visual (frontend) hacia la lógica de negocio y la persistencia de datos (backend). La tienda Sara ha aprobado el diseño web previamente presentado y ahora requiere que dicho sitio sea funcional. Para lograr esto, se implementará una API REST utilizando el framework Spring Boot con Java. Una API REST (Representational State Transfer) es un estilo arquitectónico que permite la comunicación entre sistemas a través de métodos HTTP estandarizados (GET, POST, PUT, PATCH, DELETE).
El objetivo principal de esta actividad es desarrollar un CRUD (Create, Read, Update, Delete) completo para la gestión de productos. Esto permitirá a los administradores de la tienda realizar altas, bajas, modificaciones y consultas de los productos almacenados en la base de datos SQL Server de manera dinámica y eficiente. A lo largo de este documento, se detallará la configuración del entorno de desarrollo, la estructuración del proyecto en capas (Modelos, DTOs, Repositorios, Servicios y Controladores) y la implementación de las operaciones fundamentales que darán vida al sistema web de la tienda.

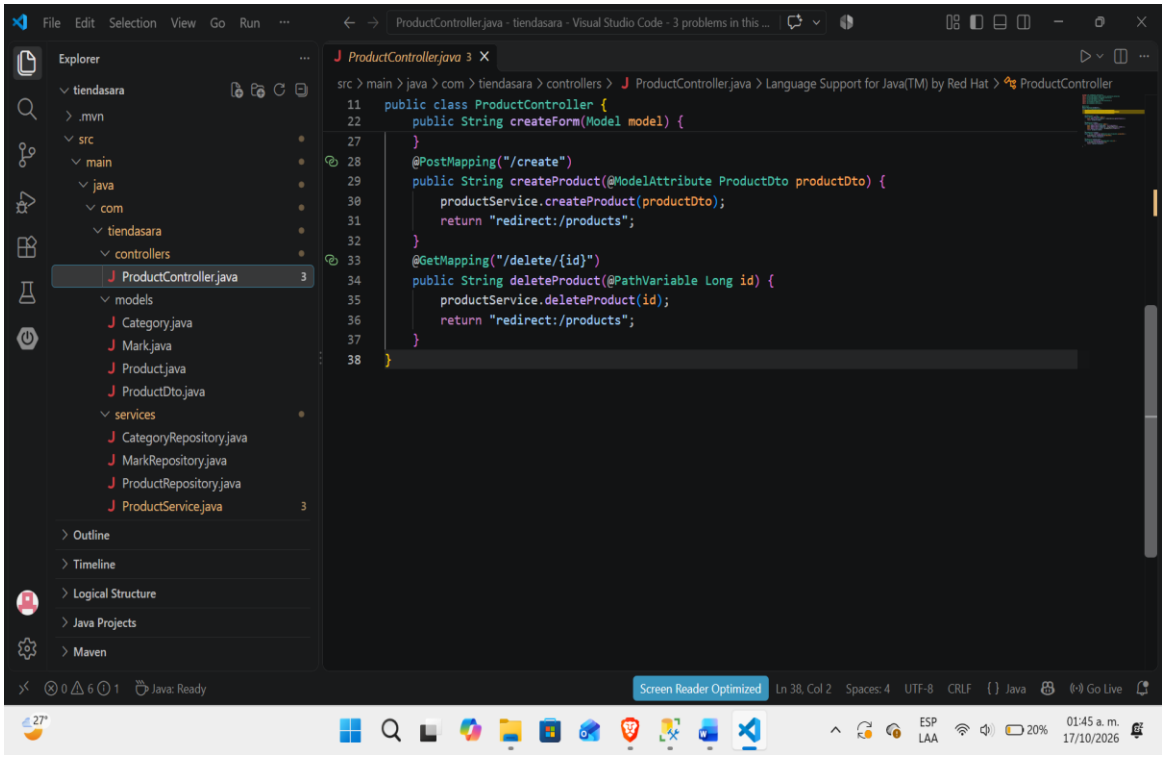
Investigación.

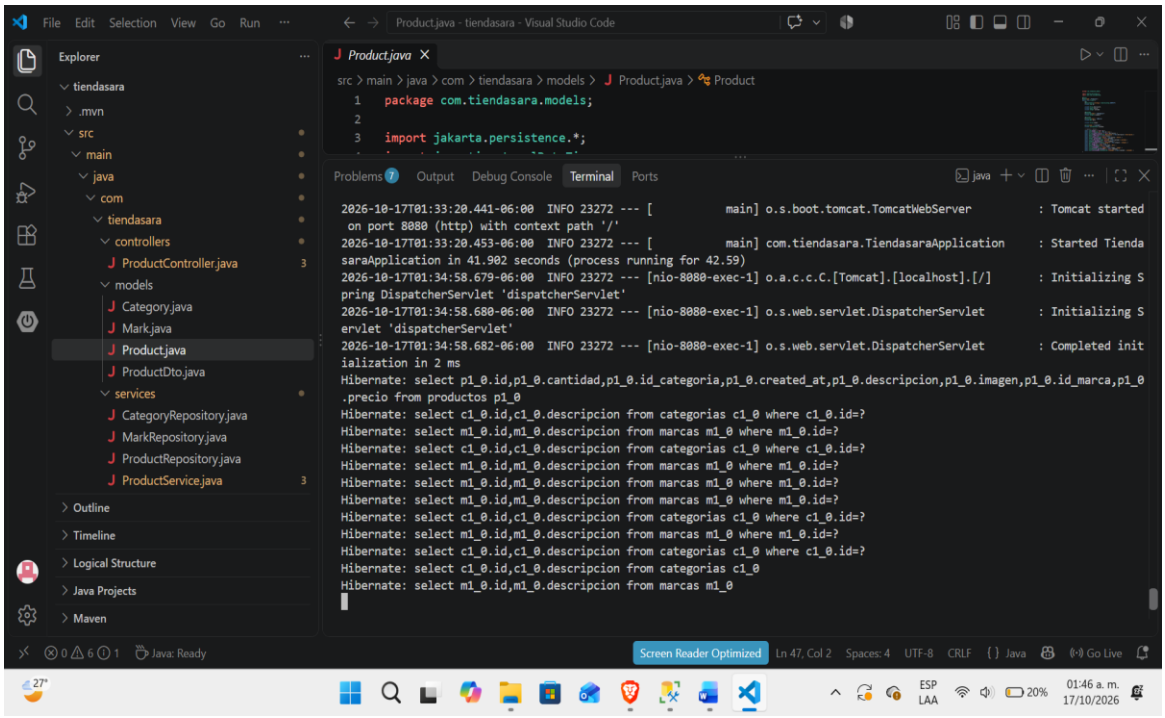
Método	Función	Descripción	Ejemplo de uso
Get	Leer/Consultar	Solicita datos de un recurso específico sin modificarlo. Es el método más común y no debe tener efectos secundarios en el servidor.	Obtener la lista completa de productos (GET /api/products) o un producto por su ID (GET /api/products/1).
Post	Crear	Envía datos al servidor para crear un nuevo recurso. Los datos van en el cuerpo (body) de la petición.	Crear un nuevo producto enviando sus datos en formato JSON al servidor (POST /api/products).
Put	Actualizar/Reemplazar	Reemplaza por completo un recurso existente con los datos enviados. Si el recurso no existe, puede crearlo (dependiendo de la implementación).	Actualizar TODOS los datos de un producto (nombre, precio, cantidad, etc.) a la vez (PUT /api/products/1).
Patch	Actualizar/Modificar	Modifica parcialmente un recurso existente. Solo se envían los campos que se quieren cambiar.	Cambiar solo el precio de un producto, sin modificar el nombre ni la cantidad (PATCH /api/products/1).
Delete	Eliminar	Borra un recurso específico del servidor.	Eliminar un producto de la base de datos por su ID (DELETE /api/products/1).

Desarrollo.
Código de la REST API sobre productos.

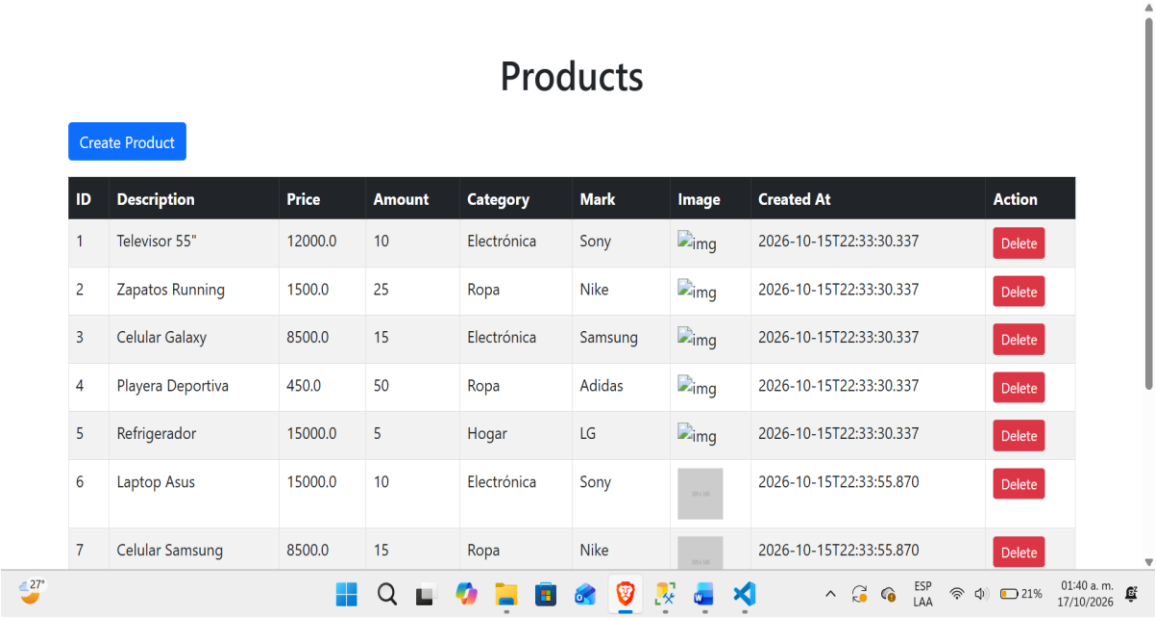


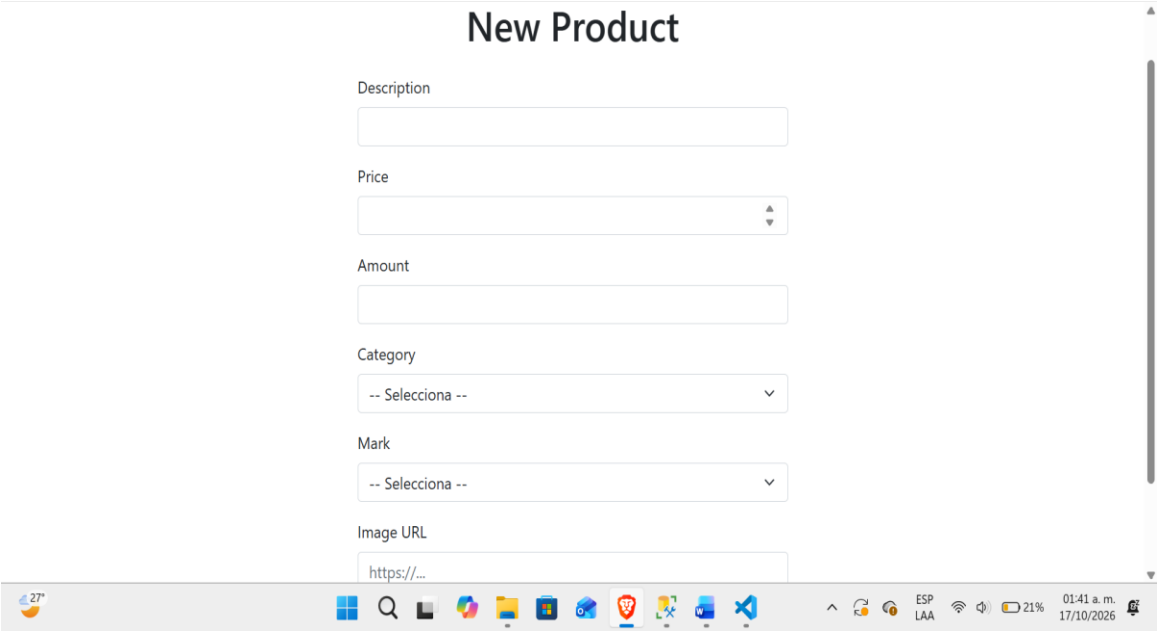



File Edit Selection View Go Run
ProductController.java - tiendasara - Visual Studio Code - 3 problems in this ...
Explorer
tiendasara
.mvn
src
main
java
com
tiendasara
controllers
ProductController.java 3
models
Category.java
Mark.java
Product.java
ProductDto.java
services
CategoryRepository.java
MarkRepository.java
ProductRepository.java
ProductService.java 3
Outline
Timeline
Logical Structure
Java Projects
Maven
ProductController.java 3
src > main > java > com > tiendasara > controllers > ProductController.java > Language Support for Java(TM) by Red Hat > ProductController
11 public class ProductController {
22 public String createForm(Model model) {
27 }
28 @PostMapping("/create")
29 public String createProduct(@ModelAttribute ProductDto productDto) {
30 productService.createProduct(productDto);
31 return "redirect:/products";
32 }
33 @GetMapping("/delete/{id}")
34 public String deleteProduct(@PathVariable Long id) {
35 productService.deleteProduct(id);
36 return "redirect:/products";
37 }
38 }
Java: Ready
Screen Reader Optimized
Ln 38, Col 2
Spaces: 4
UTF-8
CRLF
Java
Go Live
ESP
LAA
20%
01:45 a. m.
17/10/2026


File Edit Selection View Go Run
Product.java - tiendasara - Visual Studio Code
Explorer
tiendasara
.mvn
src
main
java
com
tiendasara
controllers
ProductController.java 3
models
Category.java
Mark.java
Product.java
ProductDto.java
services
CategoryRepository.java
MarkRepository.java
ProductRepository.java
ProductService.java 3
Outline
Timeline
Logical Structure
Java Projects
Maven
Product.java
src > main > java > com > tiendasara > models > Product.java > Product
1 package com.tiendasara.models;
3 import jakarta.persistence.*;
Problems 7
Output
Debug Console
Terminal
Ports
2026-10-17T01:33:20.441-06:00 INFO 23272 --- [main] o.s.boot.tomcat.TomcatWebServer : Tomcat started on port 8080 (http) with context path '/'
2026-10-17T01:33:20.453-06:00 INFO 23272 --- [main] com.tiendasara.TiendasaraApplication : Started TiendasaraApplication in 41.902 seconds (process running for 42.59)
2026-10-17T01:34:58.679-06:00 INFO 23272 --- [nio-8080-exec-1] o.a.c.c.C.[Tomcat].[localhost].[/] : Initializing Spring DispatcherServlet 'dispatcherServlet'
2026-10-17T01:34:58.680-06:00 INFO 23272 --- [nio-8080-exec-1] o.s.web.servlet.DispatcherServlet : Initializing Servlet 'dispatcherServlet'
2026-10-17T01:34:58.682-06:00 INFO 23272 --- [nio-8080-exec-1] o.s.web.servlet.DispatcherServlet : Completed initialization in 2 ms
Hibernate: select p1_0.id,p1_0.cantidad,p1_0.id_categoria,p1_0.created_at,p1_0.descripcion,p1_0.imagen,p1_0.id_marca,p1_0.precio from productos p1_0
Hibernate: select c1_0.id,c1_0.descripcion from categorias c1_0 where c1_0.id=?
Hibernate: select m1_0.id,m1_0.descripcion from marcas m1_0 where m1_0.id=?
Hibernate: select c1_0.id,c1_0.descripcion from categorias c1_0 where c1_0.id=?
Hibernate: select m1_0.id,m1_0.descripcion from marcas m1_0 where m1_0.id=?
Hibernate: select m1_0.id,m1_0.descripcion from marcas m1_0 where m1_0.id=?
Hibernate: select m1_0.id,m1_0.descripcion from marcas m1_0 where m1_0.id=?
Hibernate: select c1_0.id,c1_0.descripcion from categorias c1_0 where c1_0.id=?
Hibernate: select m1_0.id,m1_0.descripcion from marcas m1_0 where m1_0.id=?
Hibernate: select c1_0.id,c1_0.descripcion from categorias c1_0 where c1_0.id=?
Hibernate: select c1_0.id,c1_0.descripcion from categorias c1_0
Hibernate: select m1_0.id,m1_0.descripcion from marcas m1_0
Java: Ready
Screen Reader Optimized
Ln 47, Col 2
Spaces: 4
UTF-8
CRLF
Java
Go Live
ESP
LAA
20%
01:46 a. m.
17/10/2026

Página web de la REST API sobre Productos funcionando.





Conclusión.

La realización de esta actividad permitió comprender el ciclo completo del desarrollo de backend para un sistema web de comercio electrónico. Mediante la implementación de una API REST con Spring Boot, se aplicó el patrón de arquitectura MVC (Modelo-Vista-Controlador), separando correctamente las responsabilidades: los modelos para representar los datos, los repositorios para la persistencia, los servicios para la lógica de negocio y los controladores para gestionar las peticiones HTTP.

Uno de los aprendizajes más valiosos fue la configuración de la conexión a una base de datos relacional (SQL Server) y la resolución de conflictos entre los nombres de las columnas en la base de datos y los mapeos de Hibernate (JPA). Asimismo, el uso de Thymeleaf y Bootstrap demostró ser una combinación eficiente para renderizar las vistas del CRUD de productos, permitiendo que el frontend y el backend se comuniquen de manera fluida.

Finalmente, esta práctica consolidó la importancia de las herramientas de desarrollo como Maven Wrapper y Spring DevTools, las cuales facilitan la compilación y el reinicio automático del servidor. El resultado es un sistema funcional capaz de realizar altas, bajas, consultas y modificaciones en tiempo real, sentando las bases técnicas para el desarrollo de microservicios más complejos y robustos en el ámbito profesional.

Referencias.

- Baeldung. (2024). *Spring Boot Tutorial*. https://www.baeldung.com/spring-boot
- Fowler, M. (2002). *Patterns of Enterprise Application Architecture*. Addison-Wesley.
- https://github.com/ManuelMarin14/Manuel_Marin_Act2.git
- Spring. (2024). *Spring Boot Reference Documentation*. https://docs.spring.io/spring-boot/docs/current/reference/html/
- Walls, C. (2022). *Spring Boot in Action*. Manning Publications.
- W3Schools. (2024). *HTTP Request Methods*. https://www.w3schools.com/tags/ref_httpmethods.asp